```
> # ================================================================
  ======
  #  QmSp-TBend.mpl
  #  Quantum Space-Time Bend of Special Relativity in Riemann

    #  Multidimensional Space  --  Maple 2025 transcript (Physics
    package)
  #
  #  TauTimp (Temporal Undulatory Theory) Project
  #  Author : Viorel-Mihai Popescu
  #  Companion to : QmSp-TBend.docx  /  QmSp-TBend_Derivation.md
  #
  #  Structure
  #    Section 0 : Constants and parameters
  #    Section 1 : Onduscular space wave equation (Eq. 1 and Eq. 3)

    #    Section 2 : Separation of variables + Airy / Bessel-1/3
    reduction
  #    Section 3 : Eigenvalue quantization and eigenfunction plots

    #    Section 4 : Schwarzschild metric + Einstein field
    equations (Eq. 4-5)

    #    Section 5 : TransformCoordinates -> metric (5.1), vacuum
    check

    #    Section 6 : Inverse transformation -> metric (6), Ricci
    scalar, Lambda_p

    #    Section 7 : Torsion particle P_To -- second field-
    equation variant

    #    Section 8 : Attraction / repulsion transition profile
    around 2R
  #

    #  Each computational section is followed by a structured plot
    block.
  #
  #  NOTE on the metric definition (Section 4 onward):
  #    Physics:-Setup does NOT accept  metric = diagonal([...]).
  #    A metric must be supplied either

    #      (a) as a quadratic form (line element) in the
```

```
   differentials d_(x), or
#       (b) as a set of component equations  (i,j) = expression.
#     The line-element (quadratic-form) syntax is used below.

   # ============================================================
   =========

restart;
interface(rtablesize = 12):
with(plots):



   # ============================================================
   =========
#  SECTION 0 -- CONSTANTS AND PARAMETERS

   # ============================================================
   =========
#  Physical constants (SI) and the TauTimp scale parameters.

c_light := 299792458:
   # speed of light  [m/s]
G_grav  := 6.6743e-11:
   # Newton constant [m^3 kg^-1 s^-2]
M_sun   := 1.9885e30:                          # solar mass      [kg]

R_Tau   := 3.9228e22:
   # TauTimp characteristic radius [m]
ratio_RsR := 1979.8:                           # R_S / R
R_S_scale := R_Tau * ratio_RsR / 2:
   # muon-sector scale R_S [m]

# nucleon / quark structural ratio used to fix 2*R_S
dim_nucleon := 0.841419:
   # proton charge radius [fm]
dim_quark   := 0.43:
   # quark confinement radius [fm]
twoR_S_struct := R_Tau * (dim_nucleon/dim_quark) * 2:   # [m]

# couplings
K_coupling  := 8*Pi*G_grav / c_light^4:
   # Einstein coupling  [N^-1]
```

```
Kp_coupling := G_grav / c_light^2 :
   # torsion coupling   [m/kg]

printf("R (TauTimp)         = %.4e m\n", R_Tau);
printf("R_S (muon scale)    = %.4e m\n", R_S_scale);
printf("2R  transition      = %.4e m\n", 2*R_Tau);
printf("K   = 8 Pi G / c^4 = %.6e N^-1\n", evalf(K_coupling));
printf("K_p = G / c^2       = %.6e m/kg\n", evalf(Kp_coupling));



   # ============================================================
   =========
#  SECTION 1 -- ONDUSCULAR SPACE WAVE EQUATION

   # ============================================================
   =========

   #  Eq. (1)  muon-neutrino space equation :  (R_S/r) Delta f =
   (1/c^2) f_tt

   #  Eq. (3)  electronic-neutrino gravity  :  (R  /r) Delta f =
   (1/c^2) f_tt
#  Spherical-symmetry Laplacian : Delta f = f_rr + (2/r) f_r

Delta_sph := (F, rr) → diff(F, rr, rr) + (2/rr)*diff(F, rr) :

# Eq. (1) -- space sector
PDE_space := (R_S_scale/r) * Delta_sph(f(r, t), r)
           = (1/c_light^2) * diff(f(r, t), t, t);

# Eq. (3) -- gravity sector (canonical TauTimp equation)
PDE_grav  := (R_Tau/r) * Delta_sph(f(r, t), r)
           = (1/c_light^2) * diff(f(r, t), t, t);


   # Scaling check : substitution r -> r/1979.8 maps Eq.(3) onto
   Eq.(1)
scaling_check := simplify(
    eval( (R_Tau/(r/ratio_RsR)) * Delta_sph(g(r), r) )
    - (R_S_scale/r) * Delta_sph(g(r), r)
);
printf("Eq.(3) -> Eq.(1) scaling residual = %a  (expect 0)\n",
    scaling_check);
```

```
   # ============================================================
   =========
#  SECTION 2 -- SEPARATION OF VARIABLES + AIRY / BESSEL REDUCTION

   # ============================================================
   =========
#  f(r,t) = g(r) u(t).  Separation constant -k^2.

# temporal factor : harmonic oscillator
ODE_time := diff(u(t), t, t) + c_light^2 * k^2 * u(t) = 0:
sol_time := dsolve(ODE_time, u(t));

# radial equation (3a) :  g'' + (2/r) g' + (k^2/R) r g = 0
ODE_radial := diff(g(r), r, r) + (2/r)*diff(g(r), r)
              + (k^2/R_Tau)*r*g(r) = 0:
sol_radial := dsolve(ODE_radial, g(r));


   # reduction g = psi/r  -> Airy-type equation (3b) : psi'' +
   (k^2/R) r psi = 0
ODE_psi := diff(psi(r), r, r) + (k^2/R_Tau)*r*psi(r) = 0:
sol_psi := dsolve(ODE_psi, psi(r));
   # AiryAi / AiryBi solution

print("Temporal solution :", sol_time);
print("Airy-form solution:", sol_psi);
print("Radial solution   :", sol_radial);



   # ============================================================
   =========
#  SECTION 3 -- EIGENVALUE QUANTIZATION AND EIGENFUNCTION PLOTS

   # ============================================================
   =========
#  Boundary condition g(2 R_S) = 0  selects the spectrum.
#  a_n are the negative zeros of AiryAi.

Nmodes := 5:

# build the list of the first Nmodes negative Airy zeros
```

```
   # (fsolve is bracketed mode by mode; result collected with
   [seq(...)])
airy_zeros := [ seq( fsolve( AiryAi(x) = 0, x = -2.4*nn ..
   -0.05 ), nn = 1 .. Nmodes ) ]:
print("First Airy zeros a_n :", airy_zeros);

# quantization : k_n^2 = R * |a_n|^3 / (2 R_S)^3
k_sq := nn -> R_Tau * abs( airy_zeros[nn] )^3 / (2*R_S_scale)^3:

for nn from 1 to Nmodes do
      printf("mode %d : a_n = %.5f , k_n^2 = %.5e , omega_n =
   %.5e\n",
         nn, airy_zeros[nn], k_sq(nn), c_light*sqrt( k_sq(nn) ))
   ;
end do:


   #   PLOT 3.1 : Airy eigenstate psi(xi) = AiryAi(xi) over the
   first five wells
plot_eig_psi := plot( AiryAi(xi), xi = airy_zeros[Nmodes] .. 0,
    title  = "PLOT 3.1  Airy eigenstate psi(xi) = AiryAi(xi)",
    labels = ["xi", "psi"], thickness = 2, color = blue ):


   #   PLOT 3.2 : radial eigenfunctions g_n(r) = AiryAi(a_n * r)
   /r  (normalized r)
g_mode := (nn, rn) -> AiryAi( airy_zeros[nn] * rn ) / rn:
plot_eig_g := plot( [ g_mode(1, rn), g_mode(2, rn), g_mode(3,
    rn) ],
    rn = 0.02 .. 1,
    title
   = "PLOT 3.2  Radial eigenfunctions g_n(r) = AiryAi(a_n r)/r",
    labels = ["r / 2R_S", "g_n"],
    legend = ["n=1", "n=2", "n=3"], thickness = 2 ):


   #   PLOT 3.3 : EOWF snapshot f_n(r, t=0) for the first three
   modes
plot_eowf := plot( [ g_mode(1, rn), g_mode(2, rn), g_mode(3,
    rn) ],
    rn = 0.02 .. 1,
    title
   = "PLOT 3.3  EOWF f_n(r,0) -- first three onduscular modes",
```

```
    labels = ["r / 2R_S", "f_n(r,0)"],
    legend = ["n=1", "n=2", "n=3"], thickness = 2 ):


   #   PLOT 3.4 : space-time surface f_1(r,t) for the fundamental
   mode
f1_surf := AiryAi( airy_zeros[1] * rn ) / rn * cos( tn ):
plot_surface := plot3d( f1_surf, rn = 0.05 .. 1, tn = 0 .. 2*Pi,
    title  = "PLOT 3.4  Fundamental EOWF surface f_1(r,t)",
    labels = ["r/2R_S", "omega_1 t", "f_1"],
    axes = boxed, style = surface, shading = zhue ):

display(plot_eig_psi);
display(plot_eig_g);
display(plot_eowf);
display(plot_surface);



   # ============================================================
   =========
#  SECTION 4 -- SCHWARZSCHILD METRIC + EINSTEIN FIELD EQUATIONS

   # ============================================================
   =========

   #  Physics package : tensor setup in coordinates (t, r, theta,
   phi).

   #  The metric is supplied as a LINE ELEMENT (quadratic form in
   d_(x)),

   #  which is the form Physics:-Setup expects.  Parameters m, G,
   c symbolic.

with(Physics):
Setup(coordinatesystems = spherical):
Coordinates(X = [t, r, theta, phi]):


   # Schwarzschild line element (Eq. 5), written as a quadratic
   form in d_(x)
ds2_Schwarz := -( r*c^2 - 2*G*m )/r                 * d_(t)^2
            +  r^2                                * d_(theta)^2
```

```
            +   r^2*sin(theta)^2                    * d_(phi)^2
            +   r*c^2/( r*c^2 - 2*G*m )              * d_(r)^2:

Setup( metric = ds2_Schwarz ):

# curvature of the standard Schwarzschild vacuum solution
RicciS_Schwarz := simplify( Ricci[scalar] ):
print("Schwarzschild Ricci scalar  R_c =", RicciS_Schwarz);
   # expect 0
   print("Schwarzschild Ricci tensor  R_mu_nu : all components
   expected 0");
# component-wise confirmation
for ii in [t, r, theta, phi] do
    print( cat("  R[", ii, ",", ii, "] = "), simplify( Ricci[ii,
    ii] ) );
end do:

# Einstein field equation coupling (numeric)
K_val := evalf( 8*Pi*G_grav/c_light^4 ):
printf("K = 8 Pi G / c^4 = %.6e N^-1\n", K_val);



   # ================================================================
   =========

   #  SECTION 5 -- TRANSFORMCOORDINATES -> METRIC (5.1), VACUUM
   CHECK

   # ================================================================
   =========
#  TR : r = 1 - 2 rho / R_S = (mG - rho c^2)/(mG)

   #  The transformed metric (5.1) is still vacuum : R_c = 0,
   Lambda = 0.
#  Metric supplied again as a quadratic form in d_(x).

TR := r = (m*G - rho*c^2)/(m*G):


   # transformed line-element (metric 5.1) -- coefficients from
   the derivation
ds2_51 := ( c^4*r - 2*m*G*c^2 + 2*m^2*G^2 )/( m*G - r*c
   ^2 )                        * d_(t)^2
```

```
        - c^6*( m*G - r*c^2 )/( m^2*G^2*( c^4*r - 2*m*G*c
    ^2 + 2*m^2*G^2 ) )       * d_(r)^2
        + ( m*G - r*c^2 )^2/( m^2*G
    ^2 )                                    * d_(theta)^2
        + ( m*G - r*c^2 )^2/( m^2*G^2 )*sin(theta)
    ^2                              * d_(phi)^2 :

Setup( metric = ds2_51 ):
RicciS_51 := simplify( Ricci[scalar] ):
print("Metric (5.1) Ricci scalar =", RicciS_51);
    # expect 0 -> vacuum
    print("Metric (5.1) field equation reduces to 0 = 0 for T=0,
    Lambda=0");



    # ============================================================
    =========
#   SECTION 6 -- INVERSE TRANSFORMATION -> METRIC (6)

    # ============================================================
    =========
#   rho = mG(1-r)/c^2  applied to (5.1) gives metric (6).
#   Here R_cp and Lambda_p are NON-zero.

# metric (6) as a quadratic form in d_(x)
ds2_6 := ( 2*m*G - r*c^2 )/r
    * d_(t)^2
        - c^6*r/( m^2*G^2*( 2*m*G - r*c
    ^2 ) )                  * d_(r)^2
        + r^2
    * d_(theta)^2
        + r^2*sin(theta)^2
    * d_(phi)^2 :

Setup( metric = ds2_6 ):

RicciS_6 := simplify( Ricci[scalar] ):
print("Metric (6) Ricci scalar R_cp =", RicciS_6);

# analytic target from the derivation
R_cp_target := 2*( c^4 - G^2*m^2 )/( c^4*r^2 ):
Lambda_p    := ( c^4 - G^2*m^2 )/( 12*c^4*r^2 ):
```

```
print("Target R_cp      =", R_cp_target);
print("Lambda_p         =", Lambda_p);
print("R_cp / 12 - Lambda_p check :", simplify( R_cp_target/12
   - Lambda_p ));   # expect 0

# re-scaled mass substitution m = 2 M / R_S
Lambda_tau := ( c^4*R_S^2 - 4*G^2*Mm^2 )/( 12*R_S^2*c^4*r
   ^2 ):
print("Lambda_tau (m = 2M/R_S) =", Lambda_tau);


   #   PLOT 6.1 : Ricci scalar R_cp(r) for the Sun (numeric m =
   M_sun)
R_cp_num := 2*( c_light^4 - G_grav^2*M_sun^2 )/( c_light^4 * rr
   ^2 ):
plot_Rcp := plot( R_cp_num, rr = 1e9 .. 1e12,
    title  = "PLOT 6.1  Ricci scalar R_cp(r) of metric (6), Sun",
    labels = ["r  [m]", "R_cp  [m^-2]"], thickness = 2, color
   = red ):

#   PLOT 6.2 : cosmological constant Lambda_p(r)
Lambda_p_num := ( c_light^4 - G_grav^2*M_sun^2 )/( 12*c_light
   ^4 * rr^2 ):
plot_Lp := plot( Lambda_p_num, rr = 1e9 .. 1e12,
    title
   = "PLOT 6.2  Cosmological constant Lambda_p(r), metric (6)",
    labels = ["r  [m]", "Lambda_p  [m^-2]"], thickness = 2, color
   = magenta ):

display(plot_Rcp);
display(plot_Lp);


   # ============================================================
   =========

   #  SECTION 7 -- TORSION PARTICLE P_To : SECOND FIELD-EQUATION
   VARIANT

   # ============================================================
   =========
#  R_mu_p - (1/2) R_cp g_mu_p + Lambda g_mu_p = -K_p T_mu_p
#  Cylindrical symmetry; coupling K_p = G/c^2.
```

```
#  Comparison of the gravity (nu) and torsion (P_To) sectors.

compare_sectors := Matrix( [
   ["Quantity", "Gravity (nu)",
                        "Torsion (P_To)"          ],
   ["Coupling", evalf(8*Pi*G_grav/c_light^4),      evalf(G_grav
   /c_light^2)  ],
   ["Symmetry", "spherical",
                           "cylindrical"          ],
   ["Lambda",   0,
                                    "Lambda_p <> 0"          ]
] ):
print(compare_sectors);


   #   PLOT 7.1 : the two coupling constants (orders of
   magnitude, log scale)
plot_couplings := logplot(
    [ [[1, evalf(K_coupling)]], [[2, evalf(Kp_coupling)]] ],
    style = point, symbol = solidcircle, symbolsize = 18,
    title
   = "PLOT 7.1  Coupling constants : K (nu) vs K_p (P_To)",
    labels = ["sector  (1 = nu , 2 = P_To)",
    "coupling magnitude"],
    legend = ["K = 8 Pi G/c^4  [N^-1]", "K_p = G/c^2  [m/kg]"] ):
display(plot_couplings);



   # ============================================================
   =========
#  SECTION 8 -- ATTRACTION / REPULSION TRANSITION AROUND 2R

   # ============================================================
   =========

   #  P_To generates attraction for r < 2R and repulsion for 2R
   < r < 2R_S.
#  Model the effective radial force term changing sign at r = 2R.

twoR   := 2*R_Tau:
   # transition scale  ~ 7.846e22 m
twoR_S := 2*R_S_scale:
   # outer boundary    ~ 16.418 Gly in m
```

```
# effective torsion force : attractive (negative) inside 2R,
   # repulsive (positive) outside, normalized to +/- 1 over the
   range
F_To := rr -> piecewise(
    rr < twoR,  -( 1 - rr/twoR ),
   # attraction
                ( rr/twoR - 1 )/( twoR_S/twoR - 1 )
   # repulsion
):

   #   PLOT 8.1 : effective torsion force profile F_To(r), full
   range
plot_force := plot( F_To(rr), rr = 0 .. twoR_S,
    title
   = "PLOT 8.1  Torsion P_To force : attraction (r<2R) /
   repulsion (r>2R)",
    labels = ["r  [m]", "F_To  (arb. units)"],
    thickness = 2, color = "DarkGreen",
    view = [ 0 .. twoR_S, -1.1 .. 1.1 ] ):

#   PLOT 8.2 : zoom near the transition scale 2R
plot_force_zoom := plot( F_To(rr), rr = 0 .. 3*twoR,
    title  = "PLOT 8.2  Transition zoom near 2R = 7.846e22 m",
    labels = ["r  [m]", "F_To  (arb. units)"],
    thickness = 2, color = "DarkGreen" ):

#   PLOT 8.3 : annotated transition point at r = 2R
trans_marker := pointplot( [ [twoR, 0] ],
    symbol = soliddiamond, symbolsize = 20, color = red ):
plot_force_annot := display( plot_force_zoom, trans_marker,
    title
   = "PLOT 8.3  Attraction/repulsion transition at r = 2R" ):

display(plot_force);
display(plot_force_zoom);
display(plot_force_annot);

   # ============================================================
   =========
```

```
# END OF TRANSCRIPT -- QmSp-TBend.mpl

# TauTimp Project : Quantum Space-Time Bend in Riemann
Multidimensional Space

# ============================================================
=========
```

```
R (TauTimp)          = 3.9228e+22 m
R_S (muon scale)     = 3.8832e+25 m
2R  transition       = 7.8456e+22 m
K   = 8 Pi G / c^4 = 2.076647e-43 N^-1
K_p = G / c^2        = 7.426160e-28 m/kg
```

$$PDE_space := \frac{3.883179720 \times 10^{25} \left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2 \left(\frac{\partial}{\partial r} f(r,t) \right)}{r} \right)}{r} = \frac{\frac{\partial^2}{\partial t^2} f(r,t)}{89875517873681764}$$

$$PDE_grav := \frac{3.9228 \times 10^{22} \left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2 \left(\frac{\partial}{\partial r} f(r,t) \right)}{r} \right)}{r} = \frac{\frac{\partial^2}{\partial t^2} f(r,t)}{89875517873681764}$$

$$scaling_check := \frac{3.883179720 \times 10^{25} \left(\left(\frac{d^2}{dr^2} g(r) \right) r + 2. \frac{d}{dr} g(r) \right)}{r^2}$$

```
Eq.(3) -> Eq.(1) scaling residual = .3883179720e26*(diff(diff(g(r),
r),r)*r+2.*diff(g(r),r))/r^2  (expect 0)
```

$$sol_time := u(t) = c_1 \sin(299792458\, k\, t) + c_2 \cos(299792458\, k\, t)$$

$$sol_radial := g(r) = \frac{c_1 \, \mathrm{BesselJ}\left(\frac{1}{3}, \frac{\sqrt{2549199551}\, k\, r^{3/2}}{15000000000000000} \right)}{\sqrt{r}} + \frac{c_2 \, \mathrm{BesselY}\left(\frac{1}{3}, \frac{\sqrt{2549199551}\, k\, r^{3/2}}{15000000000000000} \right)}{\sqrt{r}}$$

$$sol_psi := \psi(r) = c_1 \, \mathrm{AiryAi}\left(-\frac{r \left(k^2\right)^{1/3} 25491995510^{1/3}}{100000000000} \right) + c_2 \, \mathrm{AiryBi}\left(\right.$$

$$-\frac{r\left(k^2\right)^{1/3}\,25491995510^{1/3}}{100000000000}\Bigg)$$

"Temporal solution :", $u(t) = c_1 \sin(299792458\,k\,t) + c_2 \cos(299792458\,k\,t)$

"Airy-form solution:", $\psi(r) = c_1\,\text{AiryAi}\left(-\frac{r\left(k^2\right)^{1/3}\,25491995510^{1/3}}{100000000000}\right) + c_2\,\text{AiryBi}\Bigg($

$$-\frac{r\left(k^2\right)^{1/3}\,25491995510^{1/3}}{100000000000}\Bigg)$$

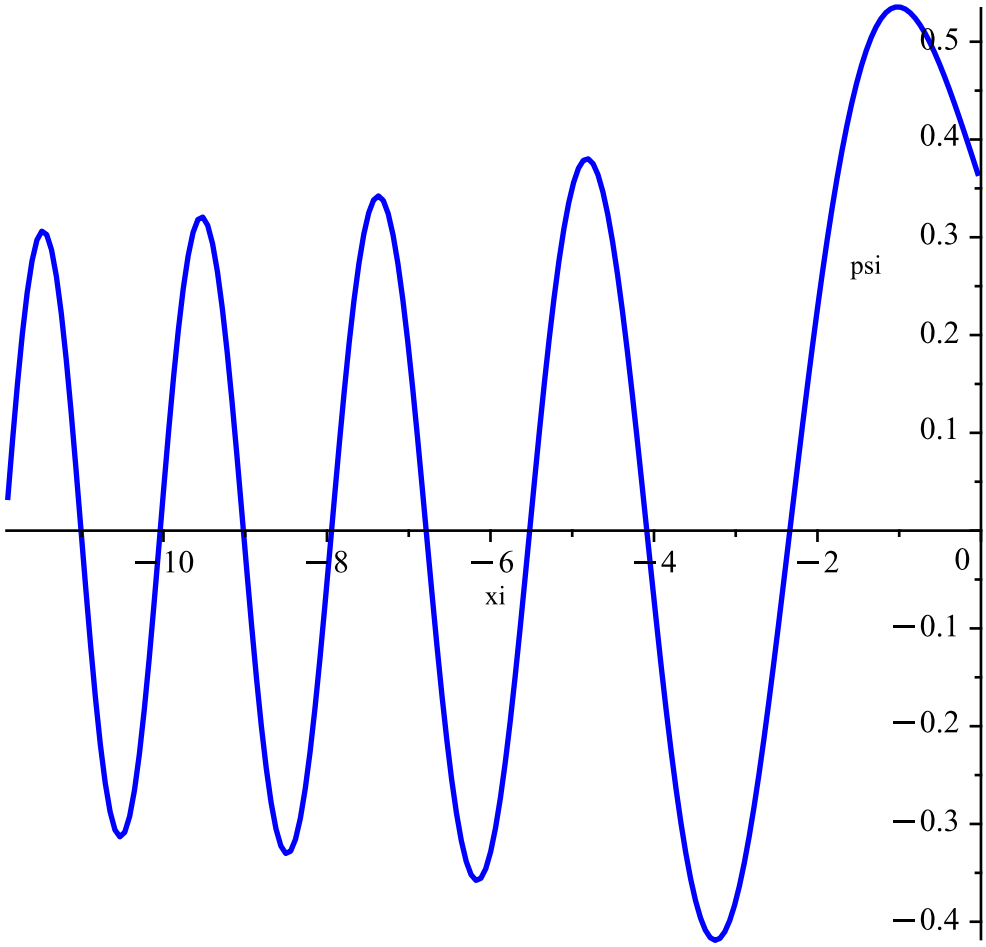
"Radial solution :", $g(r) = \dfrac{c_1\,\text{BesselJ}\left(\frac{1}{3}, \frac{\sqrt{2549199551}\,k\,r^{3/2}}{15000000000000000}\right)}{\sqrt{r}}$

$$+\frac{c_2\,\text{BesselY}\left(\frac{1}{3}, \frac{\sqrt{2549199551}\,k\,r^{3/2}}{15000000000000000}\right)}{\sqrt{r}}$$

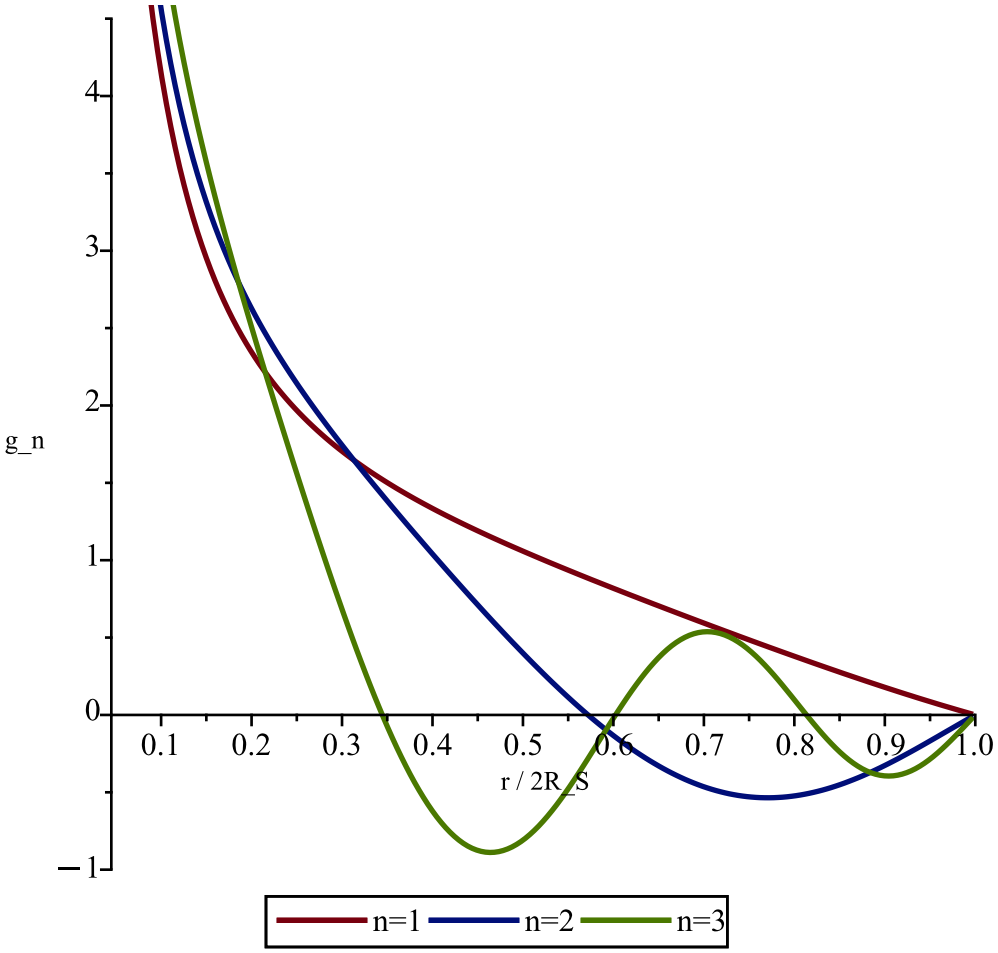
"First Airy zeros a_n :", $[-2.338107410, -4.087949444, -6.786708090, -9.022650853, -11.93601556]$

```
mode 1 : a_n = -2.33811 , k_n^2 = 1.07038e-54 , omega_n = 3.10162e-19
mode 2 : a_n = -4.08795 , k_n^2 = 5.72085e-54 , omega_n = 7.17052e-19
mode 3 : a_n = -6.78671 , k_n^2 = 2.61771e-53 , omega_n = 1.53384e-18
mode 4 : a_n = -9.02265 , k_n^2 = 6.15101e-53 , omega_n = 2.35122e-18
mode 5 : a_n = -11.93602 , k_n^2 = 1.42404e-52 , omega_n =
3.57752e-18
```

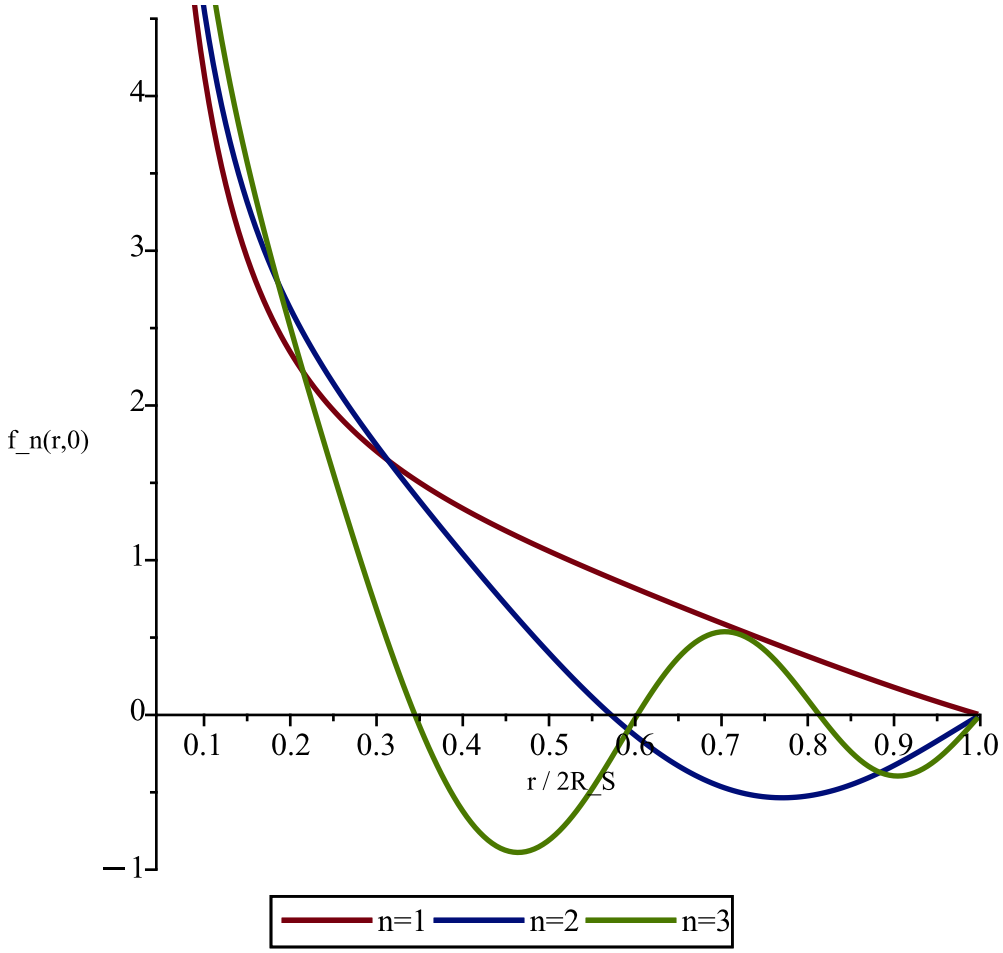
PLOT 3.1 Airy eigenstate psi(xi) = AiryAi(xi)



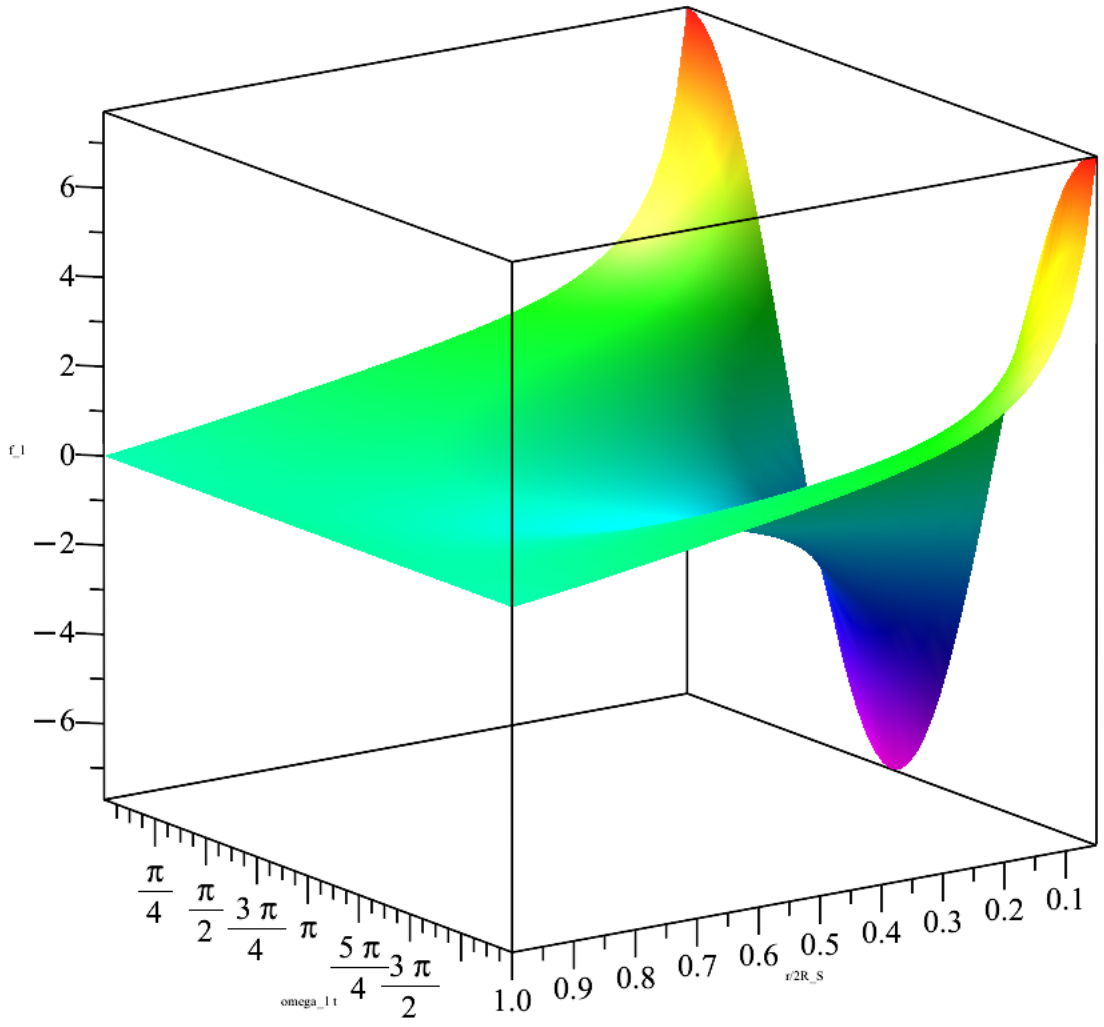
PLOT 3.2 Radial eigenfunctions g_n(r) = AiryAi(a_n r)/r



PLOT 3.3 EOWF f_n(r,0) -- first three onduscular modes



PLOT 3.4 Fundamental EOWF surface f_1(r,t)



Systems of spacetime coordinates are: $\{X = (r, \theta, \phi, t)\}$

Detected `t`, the time variable, in position 1. *Changing the signature of the spacetime metric accordingly, to* $(+ - - -)$

Systems of spacetime coordinates are: $\{X = (t, r, \theta, \phi)\}$

Coordinates: $[t, r, \theta, \phi]$. *Signature:* $(+ - - -)$

$$g_{\mu,\nu} = \begin{bmatrix} \frac{-r c^2 + 2 G m}{r} & 0 & 0 & 0 \\ 0 & \frac{r c^2}{r c^2 - 2 G m} & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin(\theta)^2 \end{bmatrix}$$

"Schwarzschild Ricci scalar R_c =", 0

"Schwarzschild Ricci tensor R_mu_nu : all components expected 0"

" R[t,t] = ", $R_{t,t}$

" R[r,r] = ", $R_{r,r}$

" R[theta,theta] = ", $R_{\theta,\theta}$

" R[phi,phi] = ", $R_{\phi,\phi}$

K = 8 Pi G / c^4 = 2.076647e-43 N^-1

Coordinates: $[t, r, \theta, \phi]$. *Signature:* $(+ - - -)$

$g_{\mu,\nu} =$

$$\begin{bmatrix} \frac{r c^4 + 2 G^2 m^2 - 2 c^2 G m}{-r c^2 + G m} & 0 & 0 & \cdots \\ 0 & -\frac{c^6 (-r c^2 + G m)}{G^2 m^2 (r c^4 + 2 G^2 m^2 - 2 c^2 G m)} & 0 & \cdots \\ 0 & 0 & \frac{(-r c^2 + \cdots}{G^2 m} & \\ 0 & 0 & 0 & \cdots \end{bmatrix}$$

"Metric (5.1) Ricci scalar =", 0

"Metric (5.1) field equation reduces to 0 = 0 for T=0, Lambda=0"

Coordinates: $[t, r, \theta, \phi]$. *Signature:* $(+ - - -)$

$$g_{\mu,\nu} = \begin{bmatrix} \frac{-r c^2 + 2 G m}{r} & 0 & 0 & 0 \\ 0 & -\frac{c^6 r}{G^2 m^2 \left(-r c^2 + 2 G m\right)} & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \sin(\theta)^2 \end{bmatrix}$$

"Metric (6) Ricci scalar R_cp =", $\frac{-2 G^2 m^2 + 2 c^4}{c^4 r^2}$

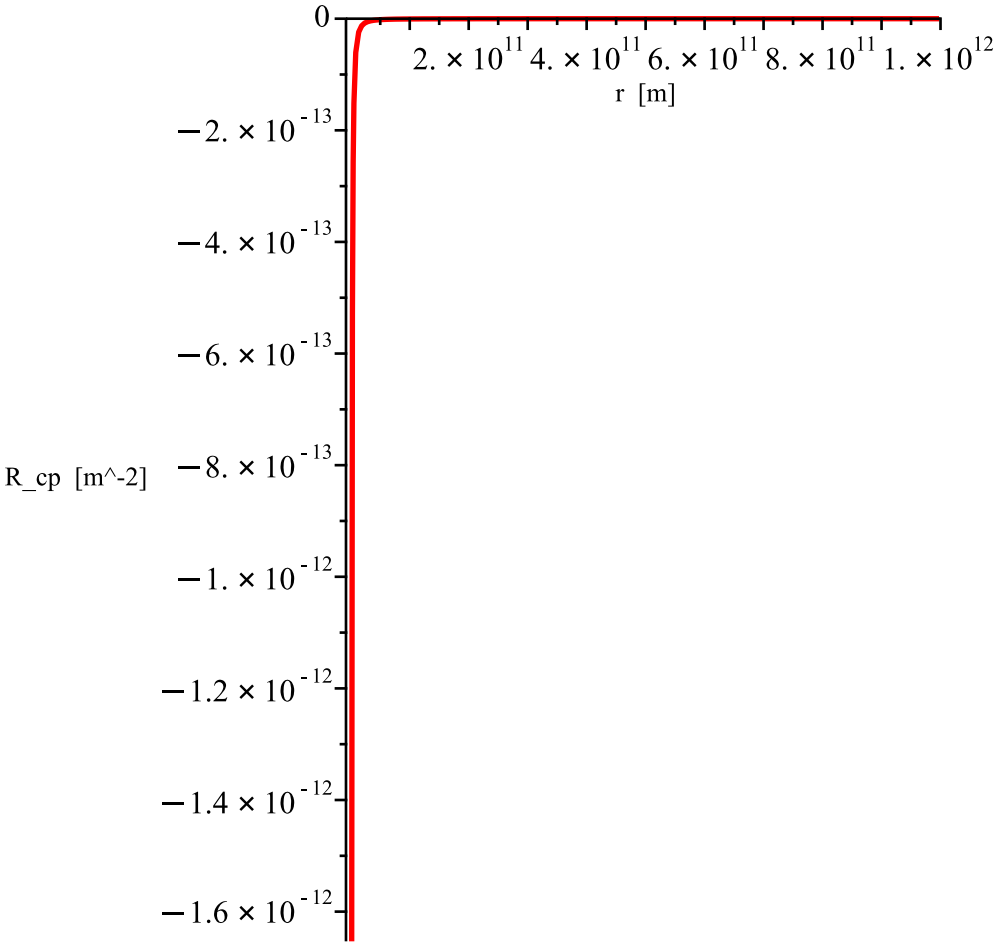
"Target R_cp =", $\frac{-2 G^2 m^2 + 2 c^4}{c^4 r^2}$

"Lambda_p =", $\frac{-G^2 m^2 + c^4}{12 c^4 r^2}$

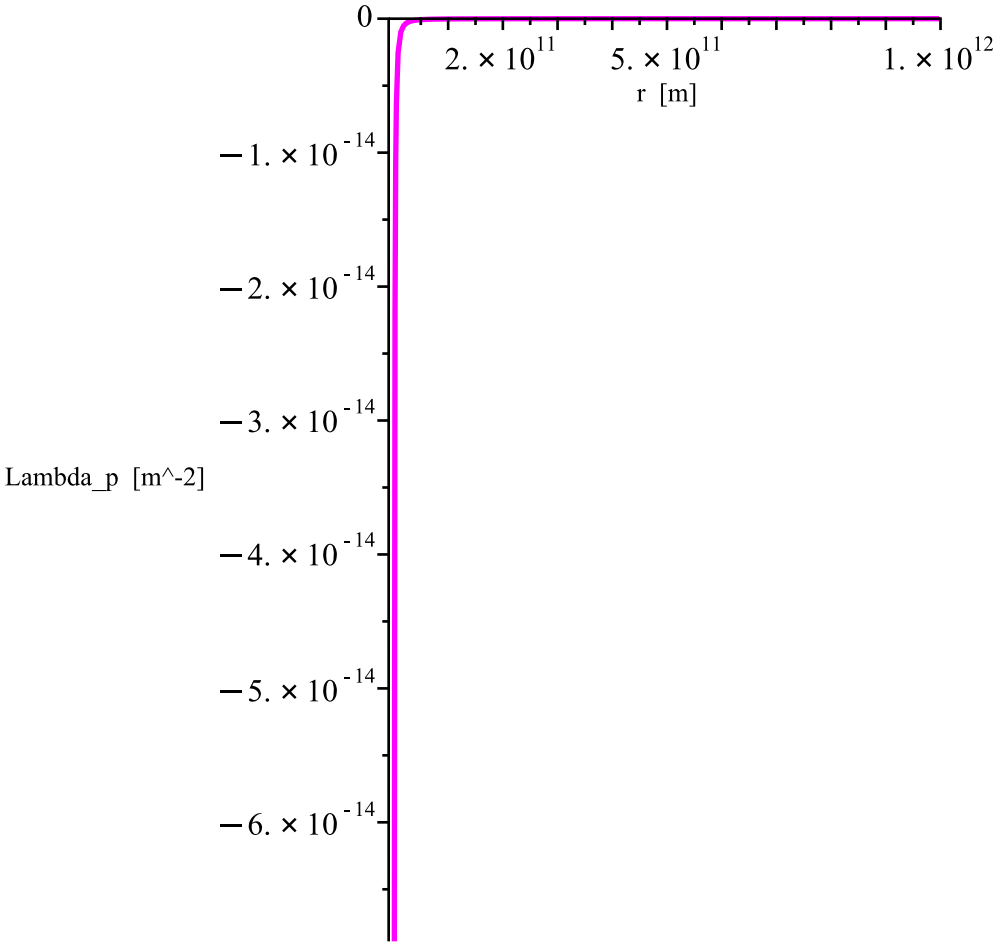
"R_cp / 12 - Lambda_p check :", $\frac{-G^2 m^2 + c^4}{12 c^4 r^2}$

"Lambda_tau (m = 2M/R_S) =", $\frac{c^4 R_S^2 - 4 G^2 M m^2}{12 c^4 R_S^2 r^2}$

PLOT 6.1 Ricci scalar R_cp(r) of metric (6), Sun

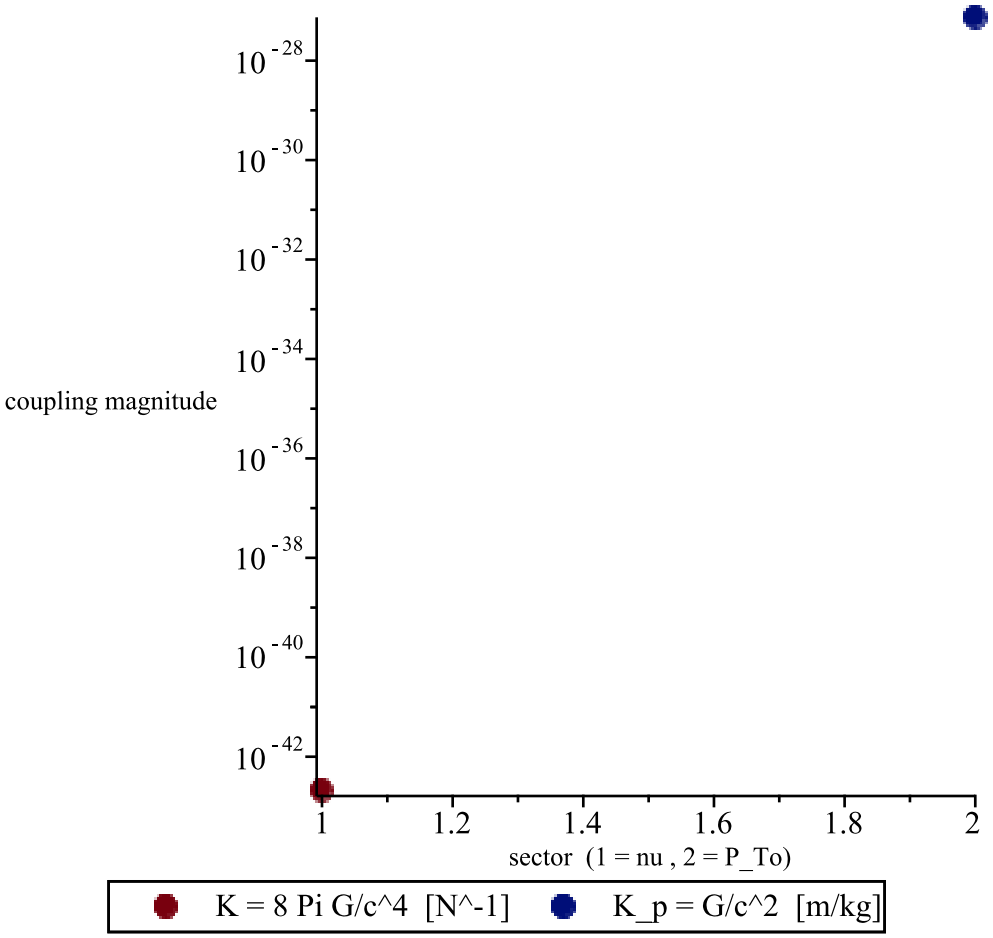


PLOT 6.2 Cosmological constant Lambda_p(r), metric (6)

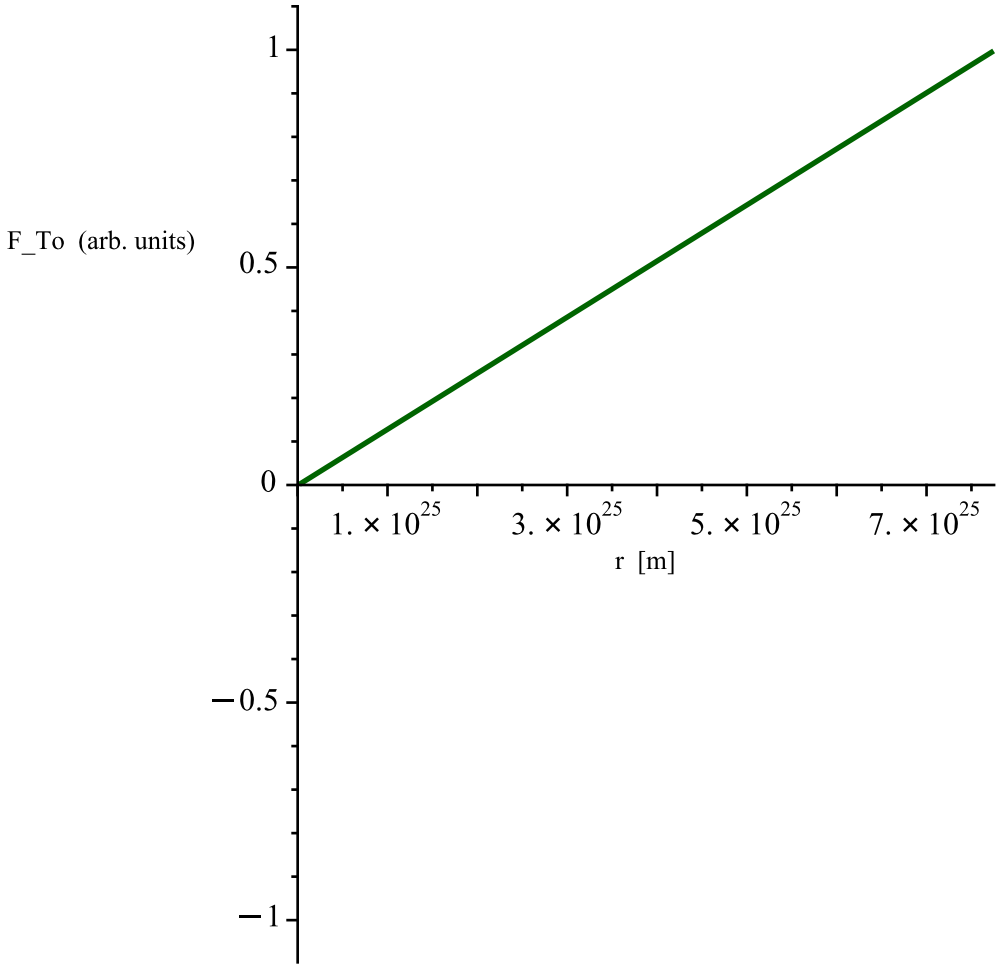


"Quantity"	"Gravity (nu)"	"Torsion (P_To)"
"Coupling"	$2.076647443 \times 10^{-43}$	$7.426160269 \times 10^{-28}$
"Symmetry"	"spherical"	"cylindrical"
"Lambda"	0	"Lambda_p <> 0"

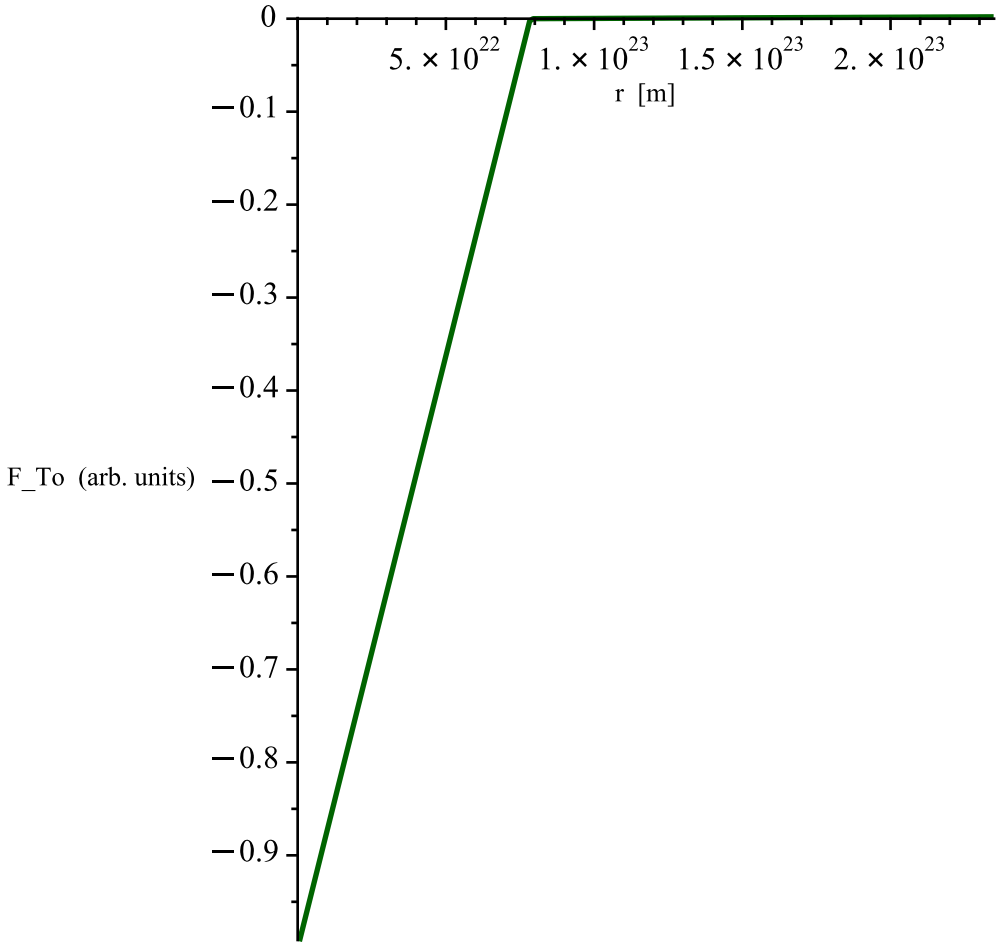
PLOT 7.1 Coupling constants : K (nu) vs K_p (P_To)

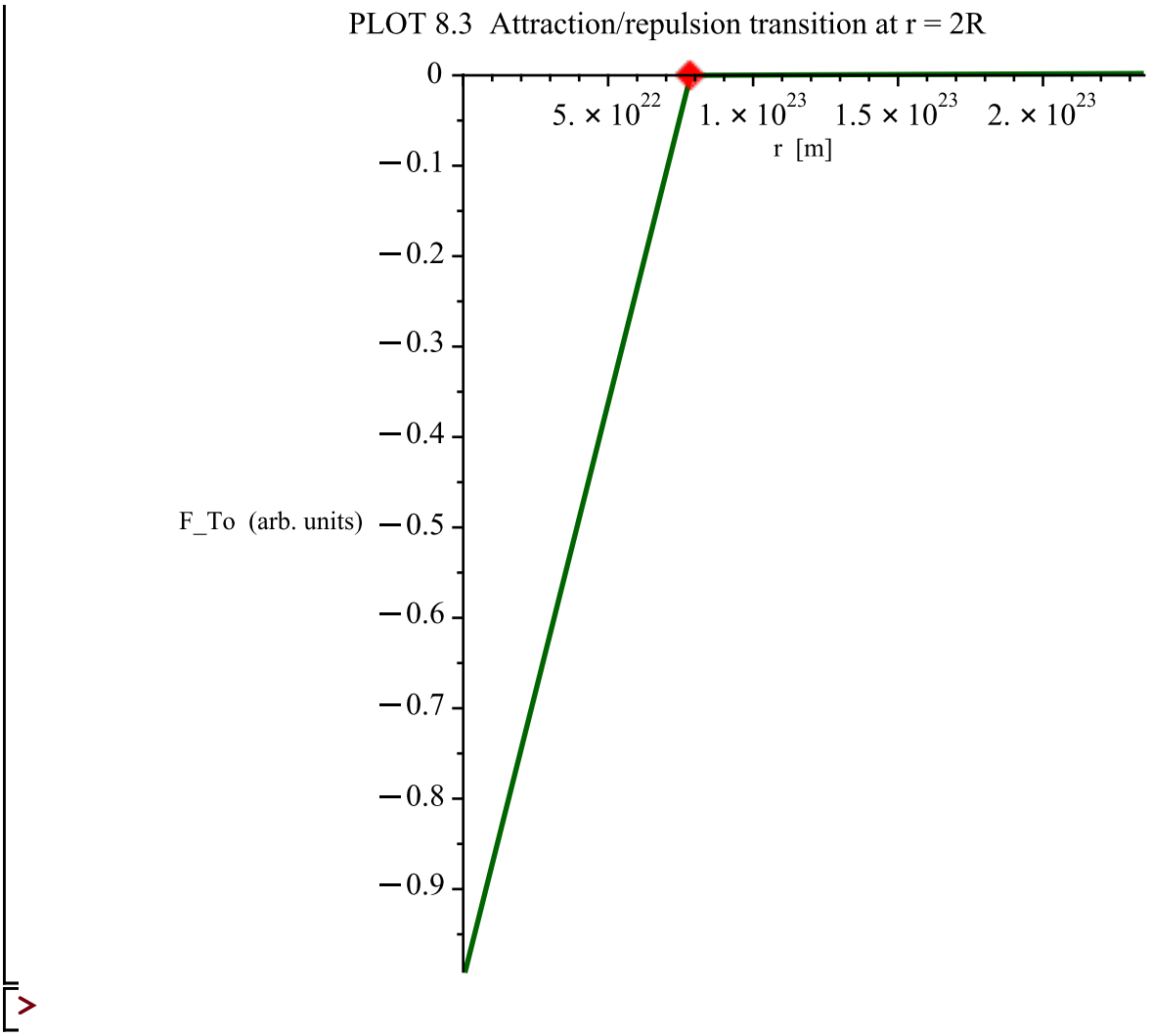


PLOT 8.1 Torsion P_To force : attraction (r<2R) / repulsion (r>2R)



PLOT 8.2 Transition zoom near 2R = 7.846e22 m




PLOT 8.3 Attraction/repulsion transition at r = 2R
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.6
−0.7
−0.8
−0.9
F_To (arb. units)
5. × 10^22
1. × 10^23
1.5 × 10^23
2. × 10^23
r [m]